

# Module 2

## Transfer Learning, Séquences & XAI

Fine-tuning, LSTM/GRU, interprétabilité Grad-CAM / SHAP / LIME / IG

### De la réutilisation à l'explication

ImageNet → tâches cibles — mémoire à long terme — audit de modèles



**Transfer**

Fine-tuning

$h_t$

**LSTM / GRU**

Portes, BPTT

$\nabla_x f$

**Grad-CAM**

Saliency

$\phi_i$

**SHAP / LIME**

Attribution

**Bassem Ben Hamed**

*bassem.benhamed@enetcom.usf.tn*

ENETCOM — Université de Sfax

Présentation 02 — Module 2

1. Transfer Learning : principe & workflow
2. Modèles récurrents : RNN, LSTM, GRU

3. Explainable AI (XAI) — pourquoi & comment
4. Synthèse & références

## **Transfer Learning : principe & workflow**

---

# Transfer Learning — formulation mathématique

## Cadre formel

Soit un modèle source  $f_{\theta_S}$  entraîné sur  $(\mathcal{X}_S, \mathcal{Y}_S) \sim \mathcal{D}_S$  (ex. ImageNet, 1.28M images, 1000 classes). On cherche un modèle cible  $f_{\theta_T}$  pour  $(\mathcal{X}_T, \mathcal{Y}_T) \sim \mathcal{D}_T$  avec  $|\mathcal{D}_T| \ll |\mathcal{D}_S|$ .

$$\theta_T = \arg \min_{\theta} \mathbb{E}_{(x,y) \sim \mathcal{D}_T} [\mathcal{L}(f_{\theta}(x), y)] \quad \text{avec init. } \theta_{\text{init}} = \theta_S.$$

## Hypothèse de transférabilité

Les **features bas/intermédiaires** (arêtes, textures, formes) sont *universelles* en vision et réutilisables. Les *features haut-niveau* sont spécifiques à la tâche source  $\rightarrow$  à remplacer.

## Quand l'utiliser ?

- Peu de données ( $10^2$ – $10^4$  images)
- Domaine proche d'ImageNet (photos naturelles)
- Budget GPU limité — entraînement  $\times 10$  à  $\times 100$  plus rapide

# Transfer Learning — deux stratégies

## Feature Extraction



`requires_grad=False` sur tout le backbone

## Fine-Tuning complet



LR différentiel : backbone  $10^{-5}$ , tête  $10^{-3}$

| Stratégie           | Quand ?                                                      | Risques                                  |
|---------------------|--------------------------------------------------------------|------------------------------------------|
| Feature extraction  | Données très limitées, domaine proche d'ImageNet, GPU faible | Capacité d'adaptation réduite            |
| Fine-tuning partiel | Quelques milliers d'exemples, domaine voisin                 | Overfitting des dernières couches        |
| Fine-tuning complet | $> 10^4$ exemples, domaine éloigné (médical, satellite)      | Catastrophic forgetting si LR trop grand |

# Workflow PyTorch — fine-tuning ResNet50

## Recette canonique (4 étapes)

1. Charger un modèle pré-entraîné depuis  
`torchvision.models`
2. Geler le backbone : `param.requires_grad = False`
3. Remplacer la tête finale par une couche adaptée à la nouvelle tâche
4. Entraîner avec un **learning rate différentiel**

## ⚠ Normalisation cohérente

Les modèles ImageNet attendent des entrées normalisées avec

$$\mu=(0.485, 0.456, 0.406), \sigma=(0.229, 0.224, 0.225)$$

⇒ utiliser ces mêmes statistiques côté cible.

```
from torchvision import models
from torchvision.models import ResNet50_Weights

model = models.resnet50(
    weights=ResNet50_Weights.DEFAULT)

# 1. Geler le backbone
for p in model.parameters():
    p.requires_grad = False

# 2. Remplacer la tete (1000 -> 2)
model.fc = nn.Linear(2048, 2)

# 3. Optimiseur a LR differentiel
opt = optim.Adam([
    {"params": model.fc.parameters(),
     "lr": 1e-3},
    {"params": model.layer4.parameters(),
     "lr": 1e-5},
], weight_decay=1e-4)
```

# Learning rate scheduling pour le fine-tuning

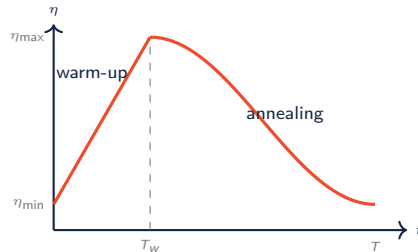
## OneCycleLR (Smith, 2018)

Cycle unique *warm-up* → *pic* → *annealing* sur la durée totale d'entraînement :

$$\eta_t = \begin{cases} \eta_{\min} + (\eta_{\max} - \eta_{\min}) \cdot \frac{t}{T_w} & t \leq T_w \\ \eta_{\min} + (\eta_{\max} - \eta_{\min}) \cdot \frac{1 + \cos\left(\pi \frac{t - T_w}{T - T_w}\right)}{2} & t > T_w \end{cases}$$

Avec  $T_w \approx 0.3 T$  (warm-up sur 30 % des steps).

**Pourquoi ça marche ?** le warm-up stabilise BN/Adam au démarrage ; le pic explore rapidement le paysage ; l'annealing fine-règle.



`optim.lr_scheduler.OneCycleLR`

# Cas d'usage — domaines médical & industriel

| Domaine                 | Modèle source               | Stratégie                                                                             | Métriques cibles          |
|-------------------------|-----------------------------|---------------------------------------------------------------------------------------|---------------------------|
| Radiologie thoracique   | ResNet50 ImageNet           | Fine-tuning des 2 derniers blocs                                                      | AUC-ROC, sensibilité      |
| Rétinopathie diabétique | EfficientNet-B4 ImageNet    | Fine-tuning complet ( 30k images)                                                     | Kappa, F1 macro           |
| Défauts industriels     | EfficientNet-B0 ou ResNet18 | Feature extraction (peu de défauts)                                                   | Précision @ rappel fixé   |
| Imagerie satellite      | ResNet50 ImageNet           | Fine-tuning complet (domaine éloigné)                                                 | mIoU, accuracy par classe |
| Dermatoscopie ISIC      | EfficientNet pré-entraîné   | TF + augmentation forte ( <a href="#">ColorJitter</a> , <a href="#">RandAugment</a> ) | Balanced accuracy         |

*Règle empirique : plus le domaine cible est éloigné d'ImageNet, plus il faut fine-tuner profondément.*



## **Modèles récurrents : RNN, LSTM, GRU**

---

# Modèles de séquences — formulation

## Cadre

Soit une séquence d'entrée  $\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_T$  avec  $\mathbf{x}_t \in \mathbb{R}^{d_{in}}$ . Un réseau récurrent maintient un **état caché**  $\mathbf{h}_t \in \mathbb{R}^{d_h}$  qui synthétise l'historique :

$$\mathbf{h}_t = \phi_{\theta}(\mathbf{h}_{t-1}, \mathbf{x}_t), \quad \mathbf{y}_t = g_{\theta}(\mathbf{h}_t).$$

Les paramètres  $\theta$  sont *partagés à travers le temps* (réutilisés à chaque pas).

## Tâches typiques en séries temporelles :

- Prédiction :  $\hat{\mathbf{x}}_{T+h}$  à partir de  $(\mathbf{x}_1, \dots, \mathbf{x}_T)$
- Détection d'anomalies : score de surprise sur l'erreur de reconstruction
- Classification de séquences : geste, état machine, qualité du processus
- Maintenance prédictive : RUL (Remaining Useful Life)

## Pourquoi pas un MLP ou un CNN 1D ?

- MLP  $\rightarrow$  taille d'entrée fixée, pas de mémoire longue
- CNN 1D  $\rightarrow$  champ réceptif borné par la profondeur
- RNN  $\rightarrow$  **contexte théoriquement infini** (en pratique : limité par le vanishing gradient)

# RNN vanille — équations, BPTT & vanishing gradient

## Équations du RNN (Elman, 1990)

$$\mathbf{h}_t = \tanh(\mathbf{W}_{xh}\mathbf{x}_t + \mathbf{W}_{hh}\mathbf{h}_{t-1} + \mathbf{b}_h)$$

$$\mathbf{y}_t = \mathbf{W}_{hy}\mathbf{h}_t + \mathbf{b}_y$$

Paramètres :  $\mathbf{W}_{xh} \in \mathbb{R}^{d_h \times d_{in}}$ ,  $\mathbf{W}_{hh} \in \mathbb{R}^{d_h \times d_h}$ ,  $\mathbf{W}_{hy} \in \mathbb{R}^{d_{out} \times d_h}$ .

## BPTT (Backpropagation Through Time)

Le gradient se propage à travers tous les pas temporels :

$$\frac{\partial \mathcal{L}_T}{\partial \mathbf{h}_t} = \frac{\partial \mathcal{L}_T}{\partial \mathbf{h}_T} \prod_{k=t+1}^T \mathbf{w}_{hh}^\top \text{diag}(\tanh'(z_k))$$

## ⚠ Vanishing / Exploding gradient

Si  $\|\mathbf{W}_{hh}^\top \text{diag}(\tanh'(\cdot))\| < 1$ , le produit s'écrase exponentiellement avec  $T$  :

$$\left\| \frac{\partial \mathcal{L}_T}{\partial \mathbf{h}_t} \right\| \sim \rho^{T-t}, \quad \rho < 1.$$

⇒ **Impossibilité d'apprendre** les dépendances longues ( $T > 50$ ).

Remède architectural : **LSTM, GRU**.

Remède numérique :

`torch.nn.utils.clip_grad_norm_` (clipping).

# LSTM — vue d'ensemble du bloc

## Idée centrale (Hochreiter & Schmidhuber, 1997)

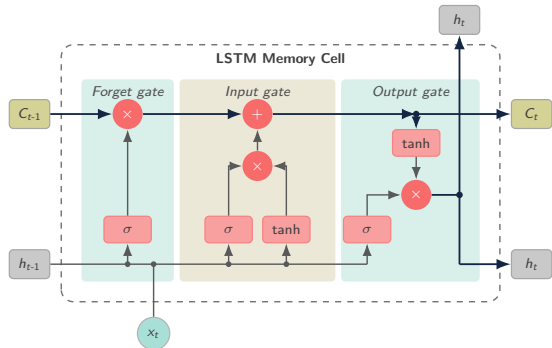
Introduire un **état cellulaire**  $c_t$  séparé de  $h_t$ , modifié par des *portes* (gates) qui contrôlent finement l'écriture, la lecture et l'oubli de l'information.

$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$$

Trois portes sigmoïdes  $\in (0, 1)^{d_h}$  :

- $f_t$  : oubli (*forget*)
- $i_t$  : entrée (*input*)
- $o_t$  : sortie (*output*)

Et un candidat de mise à jour  $\tilde{c}_t \in (-1, 1)^{d_h}$  (tanh).



Bloc LSTM : 3 portes  $\sigma$  (forget/input/output) + 1 candidat tanh, rail d'état  $C_t$

# LSTM — équations détaillées des 4 portes

## Notations

Soient  $\mathbf{x}_t \in \mathbb{R}^{d_{in}}$ ,  $\mathbf{h}_{t-1} \in \mathbb{R}^{d_h}$ . Concaténation :  $[\mathbf{h}_{t-1}, \mathbf{x}_t] \in \mathbb{R}^{d_h+d_{in}}$ . Chaque porte/candidat utilise une matrice  $\mathbf{W}_\star \in \mathbb{R}^{d_h \times (d_h+d_{in})}$  et un biais  $\mathbf{b}_\star \in \mathbb{R}^{d_h}$ .

## Porte d'oubli (forget) — que jeter ?

$$\mathbf{f}_t = \sigma(\mathbf{W}_f [\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_f)$$

$f_t^{(k)} \approx 0$  : on efface le  $k$ -ième neurone de  $\mathbf{c}_{t-1}$ .

$f_t^{(k)} \approx 1$  : on garde l'information.

## Porte d'entrée (input) — que retenir ?

$$\mathbf{i}_t = \sigma(\mathbf{W}_i [\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_i)$$

$$\tilde{\mathbf{c}}_t = \tanh(\mathbf{W}_c [\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_c)$$

$\mathbf{i}_t$  filtre les nouvelles informations candidates  $\tilde{\mathbf{c}}_t$ .

## Mise à jour de l'état cellulaire

$$\mathbf{c}_t = \underbrace{\mathbf{f}_t \odot \mathbf{c}_{t-1}}_{\text{oubli sélectif}} + \underbrace{\mathbf{i}_t \odot \tilde{\mathbf{c}}_t}_{\text{ajout sélectif}}$$

Le **constant error carousel** : si  $\mathbf{f}_t = 1$  et  $\mathbf{i}_t = 0$ , alors  $\mathbf{c}_t = \mathbf{c}_{t-1}$  et le gradient passe à 1 (pas de vanishing).

## Porte de sortie (output) — que révéler ?

$$\mathbf{o}_t = \sigma(\mathbf{W}_o [\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_o)$$

$$\mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{c}_t)$$

$\mathbf{h}_t$  est la partie de  $\mathbf{c}_t$  exposée aux couches suivantes.

# LSTM — comptage des paramètres

## Décompte par bloc LSTM

Chaque porte ( $f$ ,  $i$ ,  $o$ ) et le candidat ( $c$ ) ont une matrice ( $d_h \times (d_h + d_{in})$ ) et un biais ( $d_h$ ). Soit **4 transformations affines** :

$$\#\theta_{\text{LSTM}} = 4 \cdot [d_h(d_h + d_{in}) + d_h] = 4 d_h (d_h + d_{in} + 1).$$

## Exemple — $d_{in}=3$ (capteur 3 axes), $d_h=64$

$$4 \cdot 64 \cdot (64 + 3 + 1) = 4 \cdot 64 \cdot 68 = 17\,408 \text{ paramètres.}$$

Sur 2 couches empilées :  $\sim 51\,200$  (la 2<sup>e</sup> couche reçoit  $d_h=64$  en entrée).

## Initialisation du biais d'oubli

`nn.LSTM` initialise les biais à 0 *sauf*  $b_f$  qui peut être pré-réglé à 1 pour favoriser la mémorisation au démarrage (Jozefowicz et al., 2015).

| Composant              | Poids                  | Biais   |
|------------------------|------------------------|---------|
| Forget $f_t$           | $W_f$                  | $b_f$   |
| Input $i_t$            | $W_i$                  | $b_i$   |
| Candidat $\tilde{c}_t$ | $W_c$                  | $b_c$   |
| Output $o_t$           | $W_o$                  | $b_o$   |
| Total                  | $4 d_h (d_h + d_{in})$ | $4 d_h$ |

PyTorch concatène les 4 poids dans un seul tenseur `weight_ih_l0` (forme  $4d_h \times d_{in}$ ).

# LSTM — exemple numérique pas à pas

Configuration jouet :  $d_{in}=1$ ,  $d_h=1$ ,  $x_t = 1.0$ ,  $h_{t-1} = 0.5$ ,  $c_{t-1} = 0.2$

Poids fixés :  $W_f=W_i=W_c=W_o=[1.0, 1.0]$  (sur  $[h_{t-1}, x_t]$ ), tous biais à 0.  $\sigma$  : sigmoïde.

## Étape 1 — calcul des portes

$$z = 1.0 \cdot 0.5 + 1.0 \cdot 1.0 + 0 = 1.5$$

$$f_t = \sigma(1.5) = 0.818$$

$$i_t = \sigma(1.5) = 0.818$$

$$o_t = \sigma(1.5) = 0.818$$

$$\tilde{c}_t = \tanh(1.5) = 0.905$$

## Étape 2 — état & sortie

$$\begin{aligned} c_t &= f_t \cdot c_{t-1} + i_t \cdot \tilde{c}_t \\ &= 0.818 \cdot 0.2 + 0.818 \cdot 0.905 \\ &= 0.164 + 0.740 = 0.904 \end{aligned}$$

$$\begin{aligned} h_t &= o_t \cdot \tanh(c_t) \\ &= 0.818 \cdot \tanh(0.904) \\ &= 0.818 \cdot 0.718 = \mathbf{0.587} \end{aligned}$$

## 💡 Lecture

Avec  $f_t=0.82$ , on conserve  $\sim 82\%$  de la mémoire passée et on en ajoute  $\sim 82\%$  du nouveau candidat  $\tilde{c}_t$ . C'est le **réglage dynamique** appris par les portes : selon le contexte  $(h_{t-1}, x_t)$ , le réseau choisit d'oublier plus ou moins.

L'entraînement ajuste  $W_f, W_i, W_c, W_o$  via BPTT.

# GRU — alternative légère au LSTM

## Gated Recurrent Unit (Cho et al., 2014)

Fusionne  $c_t$  et  $h_t$ , et regroupe les portes :

$$r_t = \sigma(W_r[h_{t-1}, x_t] + b_r) \quad (\text{reset})$$

$$z_t = \sigma(W_z[h_{t-1}, x_t] + b_z) \quad (\text{update})$$

$$\tilde{h}_t = \tanh(W_h[r_t \odot h_{t-1}, x_t] + b_h)$$

$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t$$

⇒ **3 transformations affines** au lieu de 4.

| Aspect            | LSTM                     | GRU                      |
|-------------------|--------------------------|--------------------------|
| # transformations | 4                        | 3                        |
| # paramètres      | $4d_h(d_h + d_{in} + 1)$ | $3d_h(d_h + d_{in} + 1)$ |
| États maintenus   | $c_t, h_t$               | $h_t$                    |
| Mémoire longue    | Meilleure (carousel)     | Bonne                    |
| Vitesse / mémoire | Plus lourd               | Plus rapide              |

*Empiriquement, GRU = LSTM à  $\pm 1\%$  de performance pour de nombreuses tâches ; choisir GRU si latence/mémoire critique.*



# RNN bidirectionnel & bonnes pratiques d'entraînement

## Bidirectionnel : passé + futur

Deux RNN parallèles parcourent la séquence dans les deux sens :

$$\mathbf{h}_t = [\vec{\mathbf{h}}_t, \overleftarrow{\mathbf{h}}_t] \in \mathbb{R}^{2d_h}$$

Doublé le nombre de paramètres mais accède au contexte futur (ex. fenêtre glissante hors-ligne, post-traitement).

Non applicable au *forecasting en ligne* (le futur n'est pas disponible).

## Gradient clipping

$$\mathbf{g} \leftarrow \mathbf{g} \cdot \min\left(1, \frac{\tau}{\|\mathbf{g}\|_2}\right), \quad \tau \approx 1.0$$

Évite l'explosion lors des longues séquences.

```
lstm = nn.LSTM(
    input_size=3, hidden_size=64,
    num_layers=2, bidirectional=True,
    dropout=0.2, batch_first=True)

x = torch.randn(32, 100, 3)  # (B,T,F)
y, (h_n, c_n) = lstm(x)
# y.shape : (32, 100, 128)  # 2*64

# Sequences de longueurs variables :
packed = nn.utils.rnn.pack_padded_sequence(
    x, lengths, batch_first=True,
    enforce_sorted=False)
out, _ = lstm(packed)
out, _ = nn.utils.rnn.pad_packed_sequence(
    out, batch_first=True)

# Clipping pendant l'entraînement :
torch.nn.utils.clip_grad_norm_(
    lstm.parameters(), max_norm=1.0)
```

# Cas d'usage — séries temporelles industrielles

| Tâche                     | Entrée                                 | Architecture                  | Métriques       |
|---------------------------|----------------------------------------|-------------------------------|-----------------|
| Prévision conso. énergie  | Historique 24h, $T=96$ pas, 1 feature  | LSTM 2 couches $d_h=64$       | MAE, RMSE, MAPE |
| Détection d'anomalies IoT | Capteurs 6 axes, fenêtre glissante     | BiLSTM + seuil reconstruction | AUC-ROC, F1     |
| Maintenance prédictive    | Vibrations + températures (RUL)        | LSTM + tête régression        | MAE en cycles   |
| Qualité de procédé        | Mesures de production multi-canaux     | GRU + classification          | F1 macro        |
| Trafic réseau             | Volumes par minute, plusieurs services | LSTM multi-step               | sMAPE, MAE      |

## 💡 Pipeline type

Normalisation par fenêtre (z-score sur passé glissant) → BiLSTM ou GRU → tête **Linear** → loss MSE / CE selon la tâche.

## Explainable AI (XAI) — pourquoi & comment

---

## Pourquoi expliquer un modèle ?

- **Confiance** : un radiologue ne signe pas un diagnostic d'IA *boîte noire*
- **Conformité** (IA Act, FDA, RGPD art. 22) : droit à l'explication
- **Débogage** : détecter les *shortcuts* (artefact d'arrière-plan, biais)
- **Sécurité** : caractériser les modes de défaillance

| Famille              | Principe                                    | Exemples                | Coût                 |
|----------------------|---------------------------------------------|-------------------------|----------------------|
| Gradient-based       | $\nabla_x f$ et variantes spatialisées      | Saliency, Grad-CAM, IG  | Faible (1 backward)  |
| Perturbation         | Masquer / changer l'entrée, mesurer l'effet | Occlusion, RISE, LIME   | Élevé ( $N$ forward) |
| Approximation locale | Modèle linéaire interprétable autour de $x$ | LIME                    | Moyen                |
| Théorie des jeux     | Valeurs de Shapley sur features             | SHAP                    | Élevé                |
| Attention            | Cartes d'attention multi-couches            | Attention Rollout (ViT) | Faible               |

# Grad-CAM — formulation mathématique

Selvaraju et al., ICCV 2017

Soit  $f_c(x)$  le *logit* de la classe  $c$  et  $\mathbf{A}^k \in \mathbb{R}^{H \times W}$  la  $k$ -ième feature map d'une couche convolutive cible (typiquement la dernière). On calcule :

$$\alpha_c^k = \underbrace{\frac{1}{H W} \sum_{i=1}^H \sum_{j=1}^W \frac{\partial f_c}{\partial A_{ij}^k}}_{\text{poinds d'importance} \rightarrow \text{Global Average Pooling du gradient}}$$

Puis on combine *linéairement* les feature maps et on conserve les contributions positives :

$$\mathbf{L}_c^{\text{Grad-CAM}} = \text{ReLU} \left( \sum_k \alpha_c^k \mathbf{A}^k \right) \in \mathbb{R}^{H \times W}.$$

## Interprétation

$\alpha_c^k$  = importance moyenne de la  $k$ -ième feature pour la classe  $c$ . La somme pondérée donne une carte spatiale grossière ( $\sim 7 \times 7$  pour ResNet) montrant **où le réseau a regardé**.

## Variantes

- **Grad-CAM++** : gradients d'ordre 2, objets multiples
- **HiResCAM** : sans GAP  $\rightarrow$  carte plus fidèle
- **EigenCAM** : SVD des activations, sans gradient
- **FullGrad** : agrège bias-gradients toutes couches

# Grad-CAM — algorithme & code PyTorch

## Algorithme (5 étapes)

1. *Forward* jusqu'au logit cible  $f_c$
2. *Hook* sur la couche cible pour capturer  $A^k$
3. *Backward* de  $f_c$  pour obtenir  $\partial f_c / \partial A^k$
4. GAP sur l'axe spatial  $\rightarrow \alpha_c^k$
5. Combinaison linéaire + ReLU + *upsample* à la taille de l'image

## Couche cible

Pour ResNet : dernière convolution (`layer4[-1]`).

Pour un ViT : reshape de la dernière map d'attention.

```
def grad_cam(model, x, target_class, target_layer):  
    feats, grads = [], []  
    h1 = target_layer.register_forward_hook(  
        lambda m, i, o: feats.append(o))  
    h2 = target_layer.register_full_backward_hook(  
        lambda m, gi, go: grads.append(go[0]))  
  
    logits = model(x)                # (1, C)  
    score = logits[0, target_class]  
    model.zero_grad(); score.backward()  
    h1.remove(); h2.remove()  
  
    A = feats[0][0]                  # (K, H, W)  
    G = grads[0][0]                  # (K, H, W)  
    alpha = G.mean(dim=(1,2))        # (K,)  
    cam = torch.relu(  
        (alpha[:,None,None] * A).sum(0))  
    return cam / (cam.max()+1e-8)
```

# Integrated Gradients — attribution axiomatique

Sundararajan, Taly & Yan, ICML 2017

On choisit une **baseline**  $\mathbf{x}'$  neutre (image noire, moyenne, bruit). L'attribution de la  $i$ -ième feature à la prédiction  $F(\mathbf{x})$  est :

$$\text{IG}_i(\mathbf{x}) = (x_i - x'_i) \int_0^1 \frac{\partial F(\mathbf{x}' + \alpha(\mathbf{x} - \mathbf{x}'))}{\partial x_i} d\alpha.$$

En pratique (somme de Riemann à  $m$  étapes) :

$$\widehat{\text{IG}}_i(\mathbf{x}) \approx (x_i - x'_i) \frac{1}{m} \sum_{k=1}^m \frac{\partial F(\mathbf{x}' + \frac{k}{m}(\mathbf{x} - \mathbf{x}'))}{\partial x_i}.$$

## Deux axiomes garantis (uniques !)

- **Sensitivité** : si  $\mathbf{x}$  et  $\mathbf{x}'$  diffèrent sur la feature  $i$  et que  $F(\mathbf{x}) \neq F(\mathbf{x}')$ , alors  $\text{IG}_i \neq 0$
- **Complétude** :  $\sum_i \text{IG}_i = F(\mathbf{x}) - F(\mathbf{x}')$  (somme = différence totale)

## Bonnes pratiques

- Baseline : tester plusieurs (noir, gris, bruit gaussien)
- $m \in [20, 200]$  : compromis précision / coût
- Implémentation : [captum.attr.IntegratedGradients](#)
- Marche sur tout modèle différentiable (CNN, ViT, MLP)

# SHAP — valeurs de Shapley pour le ML

## Origine en théorie des jeux (Shapley, 1953)

Soit  $N = \{1, \dots, n\}$  l'ensemble des features et  $v : 2^N \rightarrow \mathbb{R}$  la valeur du modèle pour un sous-ensemble. La **valeur de Shapley** de la feature  $i$  :

$$\phi_i = \sum_{S \subseteq N \setminus \{i\}} \frac{|S|! (n - |S| - 1)!}{n!} [v(S \cup \{i\}) - v(S)].$$

*Lecture* : on moyenne la contribution marginale de  $i$  sur toutes les façons d'arriver à la coalition complète.

## Propriétés axiomatiques

- **Efficacité** :  $\sum_i \phi_i = v(N) - v(\emptyset) = F(x) - \mathbb{E}[F]$
- **Symétrie, additivité, joueur nul** (Lundberg & Lee, NeurIPS 2017)

Coût exact :  $\mathcal{O}(2^n) \rightarrow$  approximations indispensables.

## Algorithmes SHAP en pratique

- **KernelSHAP** : régression linéaire pondérée (model-agnostic)
- **DeepSHAP** / **GradientSHAP** : combine IG + Shapley (réseaux profonds)
- **TreeSHAP** : exact en temps polynomial pour les arbres
- Visualisations : *summary plot*, *force plot*, *waterfall*



# LIME — approximation linéaire locale

Ribeiro, Singh & Guestrin, KDD 2016

LIME ajuste un modèle *interprétable*  $g$  (linéaire ou arbre) approximant le comportement de  $f$  **localement autour de  $x$**  :

$$\xi(x) = \arg \min_{g \in \mathcal{G}} \mathcal{L}(f, g, \pi_x) + \Omega(g),$$

où  $\pi_x(z) = \exp(-D(x, z)^2/\sigma^2)$  pondère les voisins, et  $\Omega(g)$  pénalise la complexité.

## Pour une image (LimeImageExplainer)

1. Segmenter  $x$  en *superpixels* (slic)
2. Échantillonner  $N$  masques aléatoires  $\rightarrow$  images perturbées
3. Prédire  $f(z_i)$  pour chaque échantillon
4. Ajuster une régression **linéaire pondérée** sur l'espace des masques
5. Coefficients  $\rightarrow$  importance positive/négative de chaque superpixel

## Forces / Faiblesses

- + Totalement *model-agnostic* (boîte noire)
- + Sortie facile à présenter à un expert métier
- + Domaines hétérogènes : image, texte, tabulaire
- Instabilité : explications différentes selon le seed de sampling
- Coût élevé :  $N \in [10^3, 10^4]$  forward passes
- Choix arbitraire de  $\sigma$ , du nombre de superpixels

## Abnar & Zuidema, ACL 2020

Dans un ViT à  $L$  couches, chaque couche calcule une matrice d'attention  $\mathbf{A}^{(l)} \in \mathbb{R}^{n \times n}$  ( $n$  tokens, dont [CLS] et les patches). La connexion résiduelle se modélise par :

$$\tilde{\mathbf{A}}^{(l)} = \frac{1}{2} (\mathbf{A}^{(l)} + \mathbf{I}).$$

L'**attention cumulée** (rollout) à travers les couches :

$$\mathbf{R} = \tilde{\mathbf{A}}^{(L)} \tilde{\mathbf{A}}^{(L-1)} \dots \tilde{\mathbf{A}}^{(1)}.$$

La ligne associée au token [CLS] donne l'importance de chaque patch.

## Variantes spécifiques ViT

- **Transformer Attribution** (Chefer et al., CVPR 2021) : combine rollout + gradient
- **DINO maps** : attention auto-supervisée, segmentation émergente
- **Grad-CAM sur ViT** : appliqué à la sortie reshape de la dernière couche, marche également bien

*Comparaison empirique : Transformer Attribution > Rollout pour la précision de localisation.*

# Comparaison des méthodes XAI

| Méthode           | Type d'attribution       | Modèle requis       | Données ciblées  | Coût        | Cas d'usage           |
|-------------------|--------------------------|---------------------|------------------|-------------|-----------------------|
| Saliency          | Pixel ( $\nabla_x f$ )   | Différentiable      | Image, signal    | Très faible | Diagnostic rapide     |
| Grad-CAM          | Spatiale grossière (CNN) | Convolutif          | Image            | Faible      | Médical, défauts      |
| Integrated Grad.  | Pixel, axiomatique       | Différentiable      | Image, tabulaire | Moyen       | Audit modèle profond  |
| SHAP              | Feature (additif global) | Boîte noire ou tree | Tabulaire, image | Élevé       | Conformité, finance   |
| LIME              | Superpixel ou feature    | Boîte noire         | Image, texte     | Élevé       | Métiers, présentation |
| Attention Rollout | Patch (ViT)              | Transformer         | Image, texte     | Faible      | Interprétabilité ViT  |

## 💡 Choix selon la situation

- **Vision CNN** : Grad-CAM en première intention ; Integrated Gradients pour audit
- **Vision ViT** : Attention Rollout ou Transformer Attribution
- **Tabulaire** : SHAP (TreeSHAP si XGBoost / DeepSHAP si DL)
- **Boîte noire** (API externe) : LIME ou KernelSHAP

# Écosystème PyTorch pour la XAI

## Bibliothèques officielles

- **Captum** (Facebook AI, intégré à PyTorch) : `IntegratedGradients`, `DeepLift`, `GradientShap`, `Occlusion`, `Saliency`, `LayerGradCam`
- **pytorch-grad-cam** : Grad-CAM, HiResCAM, EigenCAM, FullGrad pour CNN et ViT
- **shap** : DeepExplainer, GradientExplainer, KernelExplainer
- **lime** : LimeImageExplainer, LimeTabularExplainer

## Bonnes pratiques d'audit

Toujours combiner **plusieurs méthodes** ; comparer aux annotations expertes ; vérifier que l'explication ne dépend pas d'artefacts (ex. étiquettes de l'hôpital sur les radios).

```
from captum.attr import IntegratedGradients
from captum.attr import LayerGradCam
```

```
# Integrated Gradients
ig = IntegratedGradients(model)
attr_ig, delta = ig.attribute(
    inputs=x,
    target=target_class,
    baselines=torch.zeros_like(x),
    n_steps=50,
    return_convergence_delta=True)
```

```
# Grad-CAM via Captum
gc = LayerGradCam(model, model.layer4[-1])
attr_gc = gc.attribute(
    inputs=x, target=target_class)
```

```
# pytorch-grad-cam (alternative)
from pytorch_grad_cam import GradCAM
cam = GradCAM(model=model,
               target_layers=[model.layer4[-1]])
grayscale_cam = cam(input_tensor=x)
```

## Synthèse & références

---

# Synthèse mathématique du Module 2

| Concept                 | Formule clé                                                      | PyTorch / outil                            |
|-------------------------|------------------------------------------------------------------|--------------------------------------------|
| Transfer feature extr.  | Gel : <code>requires_grad=False</code> ; remplacement de la tête | <code>torchvision.models.resnet50</code>   |
| LR différentiel         | $\{\eta_{\text{backbone}}=10^{-5}, \eta_{\text{head}}=10^{-3}\}$ | <code>optim.Adam([{\dots}])</code>         |
| OneCycleLR              | Warm-up linéaire $\rightarrow$ annealing cosinus                 | <code>optim.lr_scheduler.OneCycleLR</code> |
| RNN forward             | $h_t = \tanh(W_{xh}x_t + W_{hh}h_{t-1})$                         | <code>nn.RNN</code>                        |
| LSTM cell update        | $c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$                | <code>nn.LSTM</code>                       |
| GRU update              | $h_t = (1-z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t$            | <code>nn.GRU</code>                        |
| Gradient clipping       | $g \leftarrow g \cdot \min(1, \tau / \ g\ )$                     | <code>clip_grad_norm_</code>               |
| Grad-CAM                | $L_c = \text{ReLU}(\sum_k \alpha_c^k A^k)$                       | <code>captum.LayerGradCam</code>           |
| Integrated Gradients    | $(x_i - x'_i) \int_0^1 \partial F / \partial x_i d\alpha$        | <code>captum.IntegratedGradients</code>    |
| SHAP                    | $\phi_i = \sum_S w_S [v(S \cup \{i\}) - v(S)]$                   | <code>shap.DeepExplainer</code>            |
| LIME                    | $\arg \min_g \mathcal{L}(f, g, \pi_x) + \Omega(g)$               | <code>lime.LimeImageExplainer</code>       |
| Attention Rollout (ViT) | $R = \prod_l \frac{1}{2}(A^{(l)} + I)$                           | implémentation custom                      |

## Trois ouvrages couvrent les concepts présentés

1. **Atienza, R.** (2018). *Advanced Deep Learning with Keras*. Packt Publishing. ISBN 978-1-78862-941-6.  
Chapitres pertinents : RNN/LSTM, transfer learning, autoencoders.
2. **Vasilev, I.** (2019). *Python Deep Learning* (2<sup>e</sup> éd.). Packt Publishing.  
Chapitres pertinents : modèles récurrents, fine-tuning, attention.
3. **Buduma, N., Buduma, N., & Papa, J.** (2022). *Fundamentals of Deep Learning* (2<sup>e</sup> éd.). O'Reilly Media. ISBN 978-1-492-08128-7.  
Chapitres pertinents : interprétabilité (Grad-CAM, IG), séquences, optimisation.

*Les références couvrent les fondements théoriques de toutes les architectures vues dans la formation (Modules 1 à 4).*

# Du transfer à l'explication.

Cap sur le Module 3 — Autoencoders, VAE & Diffusion

✉ [bassem.benhamed@enetcom.usf.tn](mailto:bassem.benhamed@enetcom.usf.tn)